

Research Notes on Vision transformer

Vision transformer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

>> Section 1

A vision transformer (ViT) is a transformer designed for computer vision. A ViT decomposes an input image into a series of patches (rather than text into tokens), serializes each patch into a vector, and maps it to a smaller dimension with a single matrix multiplication. These vector embeddings are then processed by a transformer encoder as if they were token embeddings. ViTs were designed as alternatives to convolutional neural networks (CNNs) in computer vision applications. They have different inductive biases, training stability, and data efficiency. Compared to CNNs, ViTs are less data efficient, but have higher capacity. Some of the largest modern computer vision models are ViTs, such as one with 22B parameters. Subsequent to its publication, many variants were proposed, with hybrid architectures with both features of ViTs and CNNs. ViTs have found application in image recognition, image segmentation, weather prediction, and autonomous driving.

>> Section 2

DINO Like the Masked Autoencoder, the DINO (self-distillation with no labels) method is a way to train a ViT by self-supervision. DINO is a form of teacher-student self-distillation. In DINO, the student is the model itself, and the teacher is an exponential average of the student's past states. The method is similar to previous works like momentum contrast and bootstrap your own latent (BYOL). The loss function used in DINO is the cross-entropy loss between the output of the teacher network (f_{θ_t}) and the output of the student network (f_{θ}). The teacher network is an exponentially decaying average of the student network's past parameters: $\theta_t = \alpha \theta + (1 - \alpha) \theta_{t-1}$. The inputs to the networks are two different crops of the same image, represented as $T(x)$ and $T'(x)$, where x is the original image. The loss function is written as $L(f_{\theta_t}(T(x)), f_{\theta}(T'(x)))$. One issue is that the network can "collapse" by always outputting the same value (y), regardless of the input. To prevent this collapse, DINO employs two strategies:

>> Section 3

Instead of performing self-attention over the entire sequence of tokens, one for each patch, it performs "shifted window based" self-attention, which means only performing attention over square-shaped blocks of patches. One block of patches is analogous to the receptive field of one convolution. After every few attention blocks, there is a "merge layer", which merges neighboring 2×2 tokens into a single token. This is analogous to pooling (by 2×2 convolution kernels, with stride 2). Merging means concatenation followed by multiplication with a matrix. It is improved by Swin Transformer V2, which modifies upon the ViT by a different attention mechanism:

>> Section 4

TimeSformer The TimeSformer was designed for video understanding tasks, and it applied a factorized self-attention, similar to the factorized convolution kernels found in the Inception CNN architecture. Schematically, it divides a video into frames, and each frame into a square grid of patches (same as ViT). Let each patch coordinate be denoted by x, y, t , denoting horizontal, vertical, and time.

>> Section 5

Further reading Zhang, Aston; Lipton, Zachary; Li, Mu; Smola, Alexander J. (2024). "11.8. Transformers for Vision". Dive into deep learning. Cambridge New York Port Melbourne New Delhi Singapore: Cambridge University Press. ISBN 978-1-009-38943-3. Steiner, Andreas; Kolesnikov, Alexander; Zhai, Xiaohua; Wightman, Ross; Uszkoreit, Jakob; Beyer, Lucas (June 18, 2021). "How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers". arXiv:2106.10270 [cs.CV].

>> Section 6

The basic architecture, used by the original 2020 paper, is as follows. In summary, it is a BERT-like encoder-only Transformer. The input image is of type $R^H \times W \times C$, where H, W, C are height, width, channel (RGB). It is then split into square-shaped patches of type $R^{P \times P \times C}$. For each patch, the patch is pushed through a linear operator, to obtain a vector ("patch embedding"). The position of the patch is also transformed into a vector by "position encoding" (the paper tried no embedding, 1D embedding, 2D embedding, and relative embedding: 1D was adopted). The two vectors are added, then pushed through several Transformer encoders. The attention mechanism in a ViT repeatedly transforms representation vectors of image patches, incorporating more and more semantic relations between image patches in an image. This is analogous to how in natural language processing,

Research Notes on Vision transformer

Vision transformer

as representation vectors flow through a transformer, they incorporate more and more semantic relations between words, from syntax to semantics. The above architecture turns an image into a sequence of vector representations. To use these for downstream applications, an additional head needs to be trained to interpret them. For example, to use it for classification, one can add a shallow MLP on top of it that outputs a probability distribution over classes. The original paper uses a linear-GeLU-linear-softmax network.

>> Section 7

Training During training, input images (224px x 224 px in the original implementation) are split along a designated number of lines on each axis, producing image patches. A certain percentage of patches are selected to be masked out by mask tokens, while all others are retained in the image. The network is tasked with reconstructing the image from the remaining unmasked patches. Mask tokens in the original implementation are learnable vector quantities. A linear projection with positional embeddings is then applied to the vector of unmasked patches. Experiments varying mask ratio on networks trained on the ImageNet-1K dataset found 75% mask ratios achieved high performance on both finetuning and linear-probing of the encoder's latent space. The MAE processes only unmasked patches during training, increasing the efficiency of data processing in the encoder and lowering the memory usage of the transformer. A less computationally-intensive ViT is used for the decoder in the original implementation of the MAE. Masked patches are added back to the output of the encoder block as mask tokens and both are fed into the decoder. A reconstruction loss is computed for the masked patches to assess network performance.